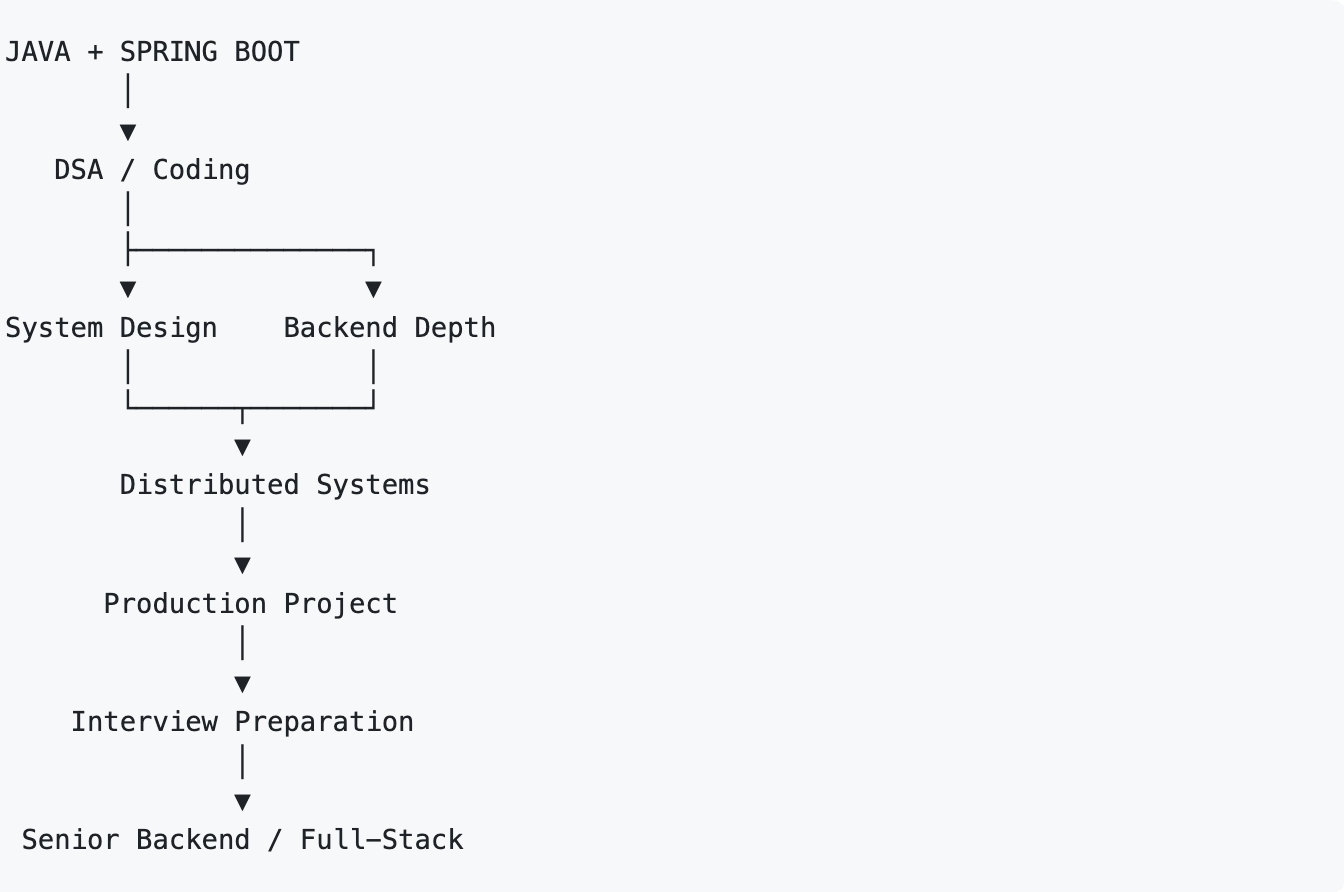
🚀 Backend /Full-Stack Career Roadmap

Goal: Transition from Senior Frontend Engineer → **Senior Backend / Backend-leaning Full-Stack Engineer**

Your existing Java + Spring Boot roadmap is now the foundation.



📚 New Roadmap

Level 11 — DSA & Problem Solving

Status: ⬜ Not Started

Focus

- Big-O analysis
- Arrays
- HashMap / HashSet
- Strings
- Two Pointers

- Two Pointers
- Sliding Window
- Prefix Sum
- Stack
- Monotonic Stack
- Queue / Deque
- Binary Search
- Linked List
- Trees
- BST
- Heap / Priority Queue
- Graphs
- BFS / DFS
- Topological Sort
- Union Find
- Recursion
- Backtracking
- Greedy
- Dynamic Programming
- Intervals

Goal

Not:

> "Solve 500 LeetCode problems."

Instead:

> **Recognize the pattern and solve a medium problem confidently in Java.**

Learning format

```
Pattern
   ↓
Intuition
   ↓
Java implementation
   ↓
Time/Space complexity
   ↓
2–4 problems
   ↓
Recognition cues
```

Level 12 — System Design Fundamentals

Status: ⬜ Not Started

Focus

- Functional vs non-functional requirements
- Scalability
- Availability
- Reliability
- Latency
- Throughput
- Load balancing
- Horizontal vs vertical scaling
- Stateless services
- Caching
- CDN
- Database selection
- SQL vs NoSQL
- Message queues
- API Gateway
- Service discovery
- Basic microservices

Core architecture

```
Client
  ↓
Load Balancer
  ↓
Application Servers
  ↓
 ┌──────────┐
 │  Cache   │
 └──────────┘
  ↓
Database
  ↓
Message Queue
  ↓
```

```
Workers
```

Design problems

Start with:

1. URL Shortener
2. Rate Limiter
3. File Storage
4. Notification System
5. Job Scheduler
6. Chat System
7. Food Delivery
8. Payment System

Level 13 — Advanced PostgreSQL & Database Engineering

Status: ⬜ Not Started

This is one of the things I would **definitely add** to the roadmap.

You already know PostgreSQL, so this should be about going deeper.

Focus

- B-Tree indexes
- Composite indexes
- Covering indexes
- Partial indexes
- Query optimization
- `EXPLAIN`
- `EXPLAIN ANALYZE`
- Sequential vs Index Scan
- Joins
- Query planner
- Transactions
- ACID
- Isolation levels

- Row locks
- Deadlocks
- Optimistic locking
- Pessimistic locking
- MVCC
- Connection pooling
- Read replicas
- Partitioning
- Sharding
- Database migrations

Important practical question

> Why is this query slow even though I have an index?

You should eventually be able to investigate this yourself.

Level 14 — Redis, Kafka & Distributed Systems

Status: ⬜ Not Started

This is another major addition.

Redis

Learn:

- Caching
- TTL
- Cache invalidation
- Cache-aside
- Distributed locks
- Counters
- Rate limiting
- Sessions
- Pub/Sub

Kafka

.

Learn:

```
Producer
   ↓
Topic
   ↓
Partition
   ↓
Consumer Group
   ↓
Consumer
```

And:

- Ordering
- Offset
- Consumer groups
- Retry
- Dead-letter queues
- At-least-once delivery
- Idempotency
- Event-driven architecture

Distributed systems

- CAP theorem
- Consistency
- Eventual consistency
- Distributed locking
- Idempotency
- Retry
- Exponential backoff
- Circuit breaker
- Bulkhead
- Timeout
- Backpressure

Level 15 — Production Spring Boot

Status: ⬜ Not Started

You already learned Spring Boot.

This level is about:

> **"Can I run and maintain this application in production?"**

Focus

- Spring Security
- JWT
- OAuth2 concepts
- RBAC
- API versioning
- Validation
- Global exception handling
- Transactions
- Async processing
- Scheduling
- Configuration management
- Profiles
- Actuator
- Logging
- Metrics
- Distributed tracing
- OpenTelemetry
- Testing
- Integration testing
- Testcontainers
- Docker

Testing pyramid

```
      E2E
       ▲
  Integration
       ▲
     Unit
```

You should be comfortable testing:

```
Controller
```

```
Service
Repository
Database integration
Authentication
Transactions
```

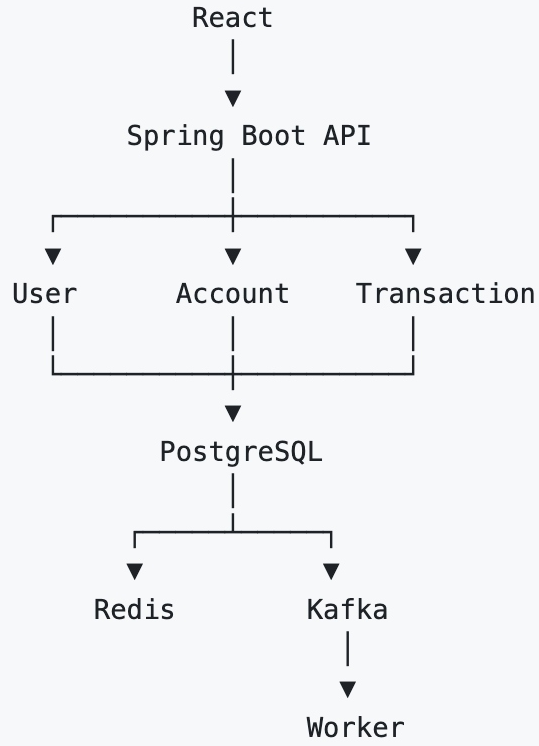
Level 16 — Production Backend Project

Status: ⬜ Not Started

This should **not** be another CRUD application.

I recommend building a serious:

Transaction / Payment Platform



Features

- Authentication
- Authorization
- Users
- Accounts
- Transfers
- Transaction history

- Idempotency
- Concurrency control
- Database transactions
- Redis caching
- Kafka events
- Retry
- Audit logs
- Pagination
- Filtering
- Rate limiting
- Docker
- Monitoring
- Tests

This becomes your **portfolio + interview project**.

Level 17 — System Design Interview Preparation

Status: ⬜ Not Started

Now move from:

> "What is Kafka?"

to:

> "Why would I use Kafka here?"

Every design should follow:

```
1. Requirements
        ↓
2. Capacity estimation
        ↓
3. API design
        ↓
4. Data model
        ↓
5. High-level architecture
        ↓
6. Detailed components
```

```
        ↓
7. Scaling
        ↓
8. Caching
        ↓
9. Async processing
        ↓
10. Failure handling
        ↓
11. Monitoring
        ↓
12. Trade-offs
```

Problems

- URL Shortener
- Instagram
- WhatsApp
- Uber
- Swiggy/Zomato
- YouTube
- Netflix
- Payment system
- Notification system
- Distributed job scheduler
- File storage system

Level 18 — Senior Backend /Full-Stack Interview Preparation

Status: ⬜ Not Started

This is the final transition stage.

DSA

- Timed problems
- Medium-heavy preparation
- Pattern recognition
- Mock coding rounds

Backend

Questions around:

- Spring Boot
- Java
- PostgreSQL
- Redis
- Kafka
- Transactions
- Concurrency
- REST
- Security
- Testing

System Design

Timed 45–60 minute designs.

Behavioral

This is particularly important because you're already senior-level.

Prepare stories around:

- Architecture decisions
- Performance improvements
- Production incidents
- Difficult technical decisions
- Mentoring
- Ownership
- Conflict
- Project failures
- Cross-team collaboration
- Technical leadership

Your existing frontend experience becomes a **strength here**, not something to hide.

⭐ One level I would add beyond the original

roadmap

Level 19 — Full-Stack Architecture & Career Positioning

Status: ⬜ Not Started

I would add this specifically because you're **not starting your career from zero**.

You're moving:

```
Senior Frontend Engineer
          ↓
Backend-capable Engineer
          ↓
Backend-leaning Full-Stack Engineer
```

So we should deliberately maintain your frontend advantage.

Focus

- React architecture
- API integration
- Authentication flows
- Frontend caching
- SSR/CSR concepts
- Web performance
- Backend-for-Frontend
- API contracts
- End-to-end architecture
- Frontend ↔ backend observability
- Full-stack deployment

The goal isn't to relearn React.

It's to be able to design:

```
Browser
   ↓
Frontend
   ↓
API Gateway
   ↓
Spring Boot
   ↓
```

```
Redis
   ↓
PostgreSQL
   ↓
Kafka
   ↓
Workers
```

end-to-end.

That is a very strong profile for you.

📊 Final Progress Tracker

```
Java + Spring Foundation
──────────────────────────────────────

Level 1  — Java Basics                ✅
Level 2  — Collections & Streams      ✅
Level 3  — OOP / Core Java            ✅
Level 4  — Exceptions / Generics      ✅
Level 5  — Modern Java                ✅
Level 6  — Concurrency                ✅
Level 7  — JVM Internals              ✅
Level 8  — Backend Fundamentals       ✅
Level 9  — Spring Boot                ✅
Level 10 — Production Backend Project ✅
```

New track

```
Level 11 — DSA                          ⬜
Level 12 — System Design Fundamentals   ⬜
Level 13 — PostgreSQL / DB Internals    ⬜
Level 14 — Redis / Kafka / Distributed  ⬜
Level 15 — Production Spring Boot       ⬜
Level 16 — Production Backend Project   ⬜
Level 17 — System Design Interviews     ⬜
Level 18 — Senior Interview Prep        ⬜
Level 19 — Full-Stack Architecture      ⬜
```

🎯 What I think you're missing right now

If I compare your completed Java/Spring journey with what a **strong senior backend candidate** needs, these are the important gaps:

Area	Current position	Priority
Java	Strong foundation	🟢
Spring Boot	Good foundation	🟢
REST APIs	Good	🟢
PostgreSQL	Intermediate	🔴
DSA	Needs structured practice	🔴
System Design	Major new area	🔴
Redis	Needs learning	🔴
Kafka	Needs learning	🔴
Distributed Systems	Needs learning	🔴
Security	Needs deeper practice	🟠
Testing	Needs production depth	🟠
Observability	Needs learning	🟠
Docker	Needs practical depth	🟠
Cloud/Deployment	Needs practical depth	🟠
Frontend	Existing strength	🟢
Behavioral/Senior interviews	Leverage existing experience	🟠

The **biggest mistake** now would be spending another month learning obscure Spring annotations while avoiding DSA and system design.

🧭 How we'll actually study

We'll keep the same learning style you've been using:

Fast-paced + practical + interview-focused.

For DSA:

```
Concept
 ↓
```

```
Pattern
 ↓
Intuition
 ↓
Java template
 ↓
Example
 ↓
Problem
 ↓
You solve
 ↓
Review
 ↓
Next pattern
```

For System Design:

```
Concept
 ↓
Real-world example
 ↓
Architecture
 ↓
Trade-offs
 ↓
Design problem
 ↓
Your solution
 ↓
Review
 ↓
Next concept
```

For Backend:

```
Concept
 ↓
Spring implementation
 ↓
Production concern
 ↓
Small feature
 ↓
Test
 ↓
Design discussion
```

And, just like your previous roadmap, **we'll create notes at the completion of each level**

rather than trying to create one enormous document upfront.

My recommendation for the immediate next step

Start Level 11 — DSA & Problem Solving.

But I would run **System Design Fundamentals in parallel from around the DSA mid-point**, rather than waiting until all DSA is finished.

That gives you the strongest path:

DSA → System Design → Backend depth → Distributed systems → Production project → Senior interviews, while preserving your existing frontend advantage